



LIONSDEV

Curso de Programação

MÓDULO 9B

Testes Automatizados da API

Professor Nicolas Cardoso Motta



Revisando o Módulo 9



Camadas

route → **controller** → **service** → **repository**:
cada uma com o seu papel.



bcrypt

hash no cadastro e **compare** no login. A senha nunca é salva nem devolvida.



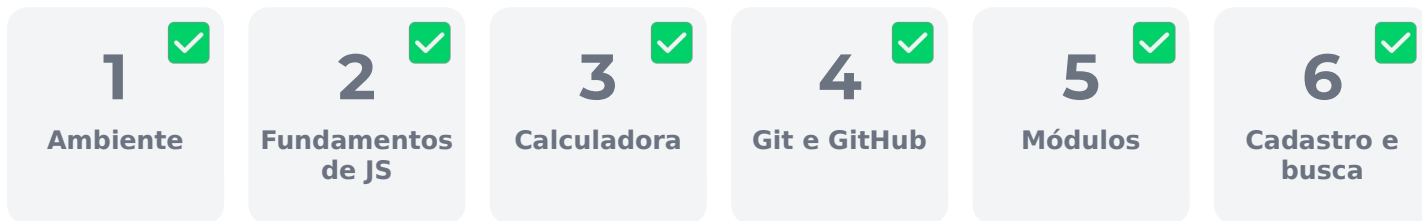
JWT

jwt.sign no login; o **middleware autenticar** confere o token nas rotas protegidas.

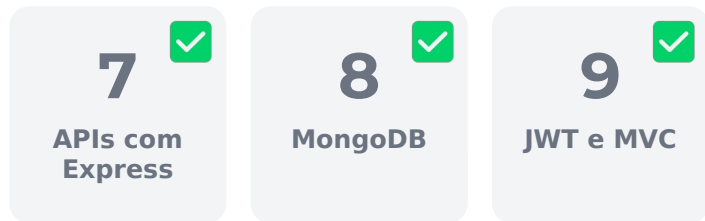
Pergunta para abrir: quantas requisições no Postman você faz para conferir o boilerplate depois de mexer no login?

Mapa do curso

BASE: JAVASCRIPT, GIT E ORGANIZAÇÃO



BACKEND



FRONTEND E PROJETO



Você está aqui: **Módulo 9B**, Testes Automatizados da API

O projeto flashcards



NO MÓDULO 9B

As rotas do flashcards ganham testes automáticos: 401 sem token, 400 sem título, 201 no cadastro. Mudou o código? `npm test` avisa se algo quebrou.



Objetivo

- Entender por que testar a API com código, e não só no Postman.
- Escrever e rodar testes com o Vitest: **describe**, **it** e **expect**.
- Testar rotas de verdade com o supertest, sem subir o servidor.
- Isolar o banco com mocks para testar cadastro, login e perfil.
- Ler um teste que falhou e escrever testes que não dependem uns dos outros.
- Documentar a API com Swagger e achar a causa de um erro com breakpoints.

No Módulo 9, cada rota foi testada **no Postman, na mão. Agora imagine mudar o login e ter que clicar em tudo de novo... **a cada mudança**.**



Por que testar com código?

O problema do teste manual



Repetitivo

Cada mudança pede refazer todas as requisições no Postman.



Fácil esquecer

Você testa o caminho feliz e esquece o 401, o 404, o e-mail repetido.



Quebra sem avisar

Arrumar uma rota pode estragar outra. Isso tem nome: regressão.



Medo de mexer

Sem testes, ninguém quer melhorar um código que "está funcionando".

Postman x teste automatizado

No Postman (manual)

1. Abrir o Postman
2. POST /api/auth/cadastro
3. Montar o body sem o nome
4. Clicar em Send
5. Conferir o status com os olhos
6. Repetir para cada caso...

No teste (automático)

```
it('sem nome responde 400', async () => {  
  const res = await request(app)  
    .post('/api/auth/cadastro')  
    .send({ senha: '123456' })  
  
  expect(res.status).toBe(400)  
})
```

O teste faz o que você faz no Postman, mas **sozinho**, em segundos, quantas vezes for preciso.

Três tipos de teste



Unitário

Testa uma função sozinha, como o **criarErro**. Rápido e simples.



De integração

Testa uma rota inteira: middlewares, controller e service juntos. O foco desta aula.



Ponta a ponta

Testa o sistema todo: front, API e banco de verdade. Mais lento e mais frágil.

Quanto mais perto do usuário, mais realista, e mais lento. Comece pelas funções e pelas rotas.



O primeiro teste com o Vitest

Instalando e configurando

No terminal: `npm install -D vitest supertest`. O `-D` instala como dependência de desenvolvimento: a API em produção não precisa deles.

```
"scripts": {  
  "start": "node src/server.js",  
  "test": "vitest run",  
  "test:watch": "vitest"  
},  
"devDependencies": {  
  "supertest": "^7.3.0",  
  "vitest": "^5.0.2"  
}
```

`test` → roda todos os testes uma vez e mostra o resultado.

`test:watch` → fica ligado e repete os testes a cada arquivo salvo.

`vitest` → entende `import` e `export` sem configuração: o boilerplate usa ES Modules.

A função que vamos testar

O `src/utils/criarErro.js` do boilerplate:

```
function criarErro(message, status) {  
  const error = new Error(message)  
  error.status = status  
  return error  
}  
  
export default criarErro
```

entrada → uma mensagem e um status.

saída → um **Error** com as duas coisas dentro.

teste → chamar a função e conferir se a saída é a esperada.

O primeiro teste

Arquivo `tests/criarErro.test.js`:

```
import { describe, it, expect } from 'vitest'
import criarErro from '../src/utils/criarErro.js'

describe('criarErro', () => {
  it('cria um erro com mensagem e status', () => {
    const erro = criarErro('Email já cadastrado.', 409)

    expect(erro.message).toBe('Email já cadastrado.')
    expect(erro.status).toBe(409)
    expect(erro).toBeInstanceOf(Error)
  })
})
```

O Vitest encontra sozinho os arquivos que terminam em `.test.js`.

describe, it e expect

describe

Agrupar os testes de um mesmo assunto: uma função, uma rota.

```
describe('criarErro', ...)
```

it

Um caso de teste. O nome diz o que deve acontecer.

```
it('nome do caso', ...)
```

expect

Confere um valor. Se não bater, o teste falha.

```
expect(valor).toBe(esperado)
```

Lendo em voz alta: "criarErro, cria um erro com mensagem e status". O relatório do teste fica assim.

Rodando: npm test

```
$ npm test
```

```
RUN v5.0.2
```

```
✓ tests/criarErro.test.js (1 test)
```

```
Test Files 1 passed (1)
```

```
Tests 1 passed (1)
```

✓ → o arquivo passou: todos os **it** dele deram certo.

Test Files → quantos arquivos de teste rodaram.

Tests → quantos casos (**it**) passaram.

Os matchers mais usados

<code>expect(erro.status).toBe(409)</code>	<code>// valor simples</code>
<code>expect([1, 2]).toEqual([1, 2])</code>	<code>// mesmo conteúdo</code>
<code>expect(res.body).toHaveProperty('token')</code>	<code>// tem esse campo</code>
<code>expect(usuario).not.toHaveProperty('senhaHash')</code>	<code>// not inverte</code>
<code>expect(res.body.token).toBeTypeOf('string')</code>	<code>// tipo do valor</code>
<code>expect(erro).toBeInstanceOf(Error)</code>	<code>// criado pela classe</code>

O `toBe` é para valores simples; para objetos e arrays, o `toEqual`. Já, já você vai ver por quê.



Testando rotas com o supertest

Por que o app.js separado ajuda

server.js: sobe o servidor

```
dotenv.config()  
await conectarBanco()  
app.listen(process.env.PORT || 3000)
```

No teste: só o app

```
import request from 'supertest'  
import app from '../src/app.js'  
  
const res = await request(app).get('/')
```

O supertest usa o app direto: sem porta, sem servidor rodando e sem banco conectado. Foi por isso que separamos o app.js do server.js no Módulo 9.

O primeiro teste de rota

```
import { describe, it, expect } from 'vitest'
import request from 'supertest'
import app from '../src/app.js'

describe('rotas básicas', () => {
  it('GET / responde 200', async () => {
    const res = await request(app).get('/')

    expect(res.status).toBe(200)
    expect(res.body.message).toBe('Boilerplate API MVC está rodando.')
  })

  it('rota que não existe responde 404', async () => {
    const res = await request(app).get('/api/nada')

    expect(res.status).toBe(404)
    expect(res.body).toEqual({ message: 'Rota não encontrada.' })
  })
})
```

request(app).get(rota) faz a requisição; o await espera a resposta.

Quando o teste falha

Troquei o **404** por **400** no teste da rota inexistente. O Vitest mostra:

```
FAIL tests/app.test.js > rotas básicas >
  rota que não existe responde 404
AssertionError: expected 404 to be 400
```

```
- Expected
+ Received
```

```
- 400
+ 404
```

```
> tests/app.test.js:16:24
```

Expected → o que o teste esperava: o valor do **toBe**.

Received → o que a API devolveu de verdade.

> → o arquivo e a linha do **expect** que falhou.

e agora? → o erro pode estar no teste ou no código: leia os dois antes de mudar.



toBe com objeto: passa ou falha?

```
it('rota que não existe responde 404', async () => {  
  const res = await request(app).get('/api/nada')  
  
  expect(res.body).toBe({ message: 'Rota não encontrada.' })  
})
```

A API responde exatamente esse body. O teste passa?



Cuidado: toBe compara o objeto, não o conteúdo

O teste **falha**, mesmo com o conteúdo igual: o **toBe** pergunta se é **o mesmo objeto**.

- Para objetos e arrays, use **toEqual**: ele compara campo a campo.
- O próprio Vitest dá a pista: **serializes to the same string** quer dizer "o conteúdo é igual, o objeto não".

```
AssertionError: expected { message: 'Rota não encontrada.' }  
  to be { message: 'Rota não encontrada.' } // Object.is equality  
Received: serializes to the same string
```

Body e headers, como no Postman

```
// body em JSON, como a aba Body do Postman
await request(app)
  .post('/api/auth/cadastro')
  .send({ email: 'ana@email.com', senha: '123456' })

// header, como a aba Headers do Postman
await request(app)
  .get('/api/usuarios/perfil')
  .set('Authorization', 'Bearer abc.def.ghi')
```

O .send() envia o objeto como JSON e já coloca o Content-Type certo. O .set() coloca um header.

Testando a validação (400)

```
import { describe, it, expect } from 'vitest'
import request from 'supertest'
import app from '../src/app.js'

describe('POST /api/auth/cadastro: validação', () => {
  it('sem nome responde 400', async () => {
    const res = await request(app)
      .post('/api/auth/cadastro')
      .send({ email: 'ana@email.com', senha: '123456' })

    expect(res.status).toBe(400)
    expect(res.body.message).toBe("O campo 'nome' é obrigatório.")
  })
})
```

O middleware validarCadastro barra antes do banco. Conferir a mensagem garante que barrou pelo motivo certo.

Testando a autenticação (401)

```
describe('GET /api/usuarios/perfil sem token válido', () => {
  it('sem token responde 401', async () => {
    const res = await request(app).get('/api/usuarios/perfil')

    expect(res.status).toBe(401)
    expect(res.body.message).toBe('Token não informado.')
  })

  it('token falso responde 401', async () => {
    const res = await request(app)
      .get('/api/usuarios/perfil')
      .set('Authorization', 'Bearer abc.def.ghi')

    expect(res.status).toBe(401)
    expect(res.body.message).toBe('Token inválido ou expirado.')
  })
})
```

Nenhum destes testes chega ao banco: o middleware autenticar responde 401 antes do controller.

Resultado até aqui

```
$ npm test
```

```
RUN v5.0.2
```

- ✓ tests/autenticacao.test.js (3 tests)
- ✓ tests/validacao.test.js (2 tests)
- ✓ tests/app.test.js (2 tests)
- ✓ tests/criarErro.test.js (1 test)

```
Test Files  4 passed (4)  
Tests      8 passed (8)
```

ordem → os arquivos rodam em paralelo: a ordem muda de uma execução para outra.

8 testes → oito requisições que você não precisa mais fazer no Postman.



Cuidado: esqueceu o await

Sem o **await**, o **res** ainda não é a resposta: **res.status** é **undefined** e o teste falha.

- O supertest devolve uma requisição a caminho: o **await** espera a resposta chegar.
- E a função do **it** precisa ser **async**, como as rotas do Módulo 8.

```
const res = request(app).get('/') // sem await
expect(res.status).toBe(200)
// AssertionError: expected undefined to be 200
```



E o banco de dados?

Cadastro, login e perfil passam pelo MongoDB...



O que acontece com este teste?

```
it('cadastra um usuário', async () => {  
  const res = await request(app)  
    .post('/api/auth/cadastro')  
    .send({ nome: 'Ana', senha: '123456',  
           email: 'ana@email.com' })  
  expect(res.status).toBe(201)  
})
```

Lembre: no teste, ninguém chamou o conectarBanco().



Cuidado: rota que usa o banco, sem banco

O teste fica **parado** e falha depois de 5 segundos: **Test timed out in 5000ms.**

- Sem conexão, o Mongoose guarda a consulta numa fila e espera o banco conectar, o que nunca acontece.
- O Vitest desiste em 5 segundos. Rota que usa o banco precisa de um banco de teste ou de um mock.

Três jeitos de lidar com o banco



Banco de teste

Outro banco no Atlas, só para testes. Realista, mas precisa de internet e de limpar os dados entre os testes.



Banco em memória

O pacote **mongodb-memory-server** sobe um MongoDB temporário no computador. Realista, mas pesado na primeira vez.

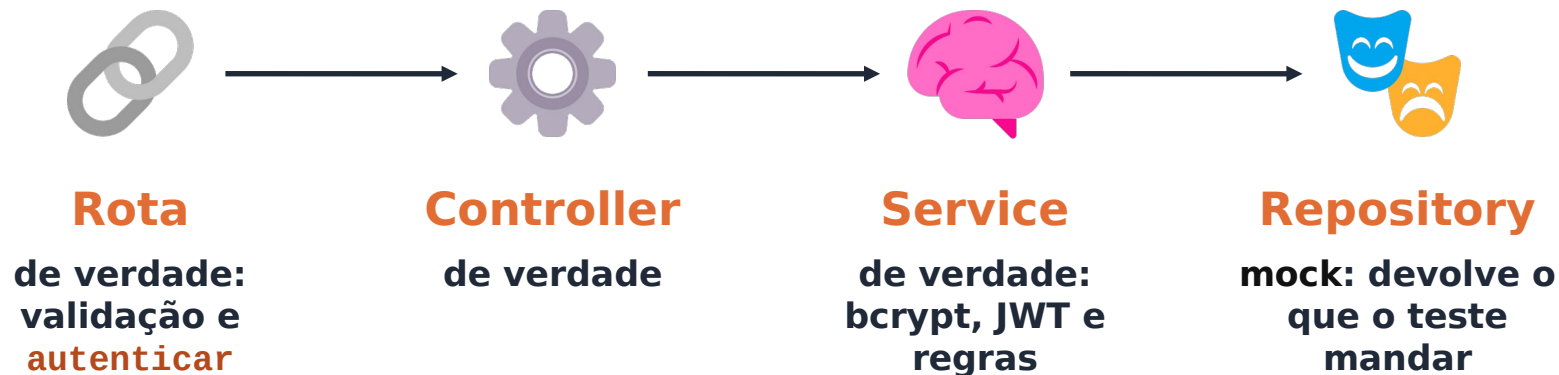


Mock

Troca o repository por uma versão de mentira. Rápido e sem banco: é o que vamos usar.

Na arquitetura em camadas, só o repository fala com o banco. Trocando ele, o resto do código roda de verdade.

O que o mock troca



O teste continua passando pela validação, pelo bcrypt e pelo JWT. Só a conversa com o MongoDB é simulada.

Criando o mock: vi.mock()

O começo do `tests/cadastro.test.js`:

```
import { describe, it, expect, vi, beforeEach } from 'vitest'
import request from 'supertest'
import app from '../src/app.js'
import Usuario from '../src/models/usuario.model.js'
import UsuarioRepository from '../src/repositories/usuario.repository.js'

vi.mock('../src/repositories/usuario.repository.js')

beforeEach(() => {
  vi.resetAllMocks()
})
```

O `vi.mock()` troca todas as funções do repository por funções vazias. O Vitest move essa linha para o topo: ela vale antes dos imports.

Cadastro com mock: 201

```
it('cadastra e não devolve a senha', async () => {
  UsuarioRepository.buscarPorEmail.mockResolvedValue(null)
  UsuarioRepository.criar.mockImplementation(
    async (dados) => new Usuario(dados)
  )

  const res = await request(app)
    .post('/api/auth/cadastro')
    .send({ nome: 'Ana', email: 'Ana@Email.com', senha: '123456' })

  expect(res.status).toBe(201)
  expect(res.body.usuario.email).toBe('ana@email.com')
  expect(res.body.usuario).not.toHaveProperty('senhaHash')
  expect(res.body.token).toBeTypeOf('string')
})
```

mockResolvedValue devolve um valor; mockImplementation roda uma função.

Cadastro com mock: e-mail repetido (409)

```
it('email repetido responde 409', async () => {  
  const ana = new Usuario({ nome: 'Ana', email: 'ana@email.com' })  
  UsuarioRepository.buscarPorEmail.mockResolvedValue(ana)  
  
  const res = await request(app)  
    .post('/api/auth/cadastro')  
    .send({ nome: 'Ana', email: 'ana@email.com', senha: '123456' })  
  
  expect(res.status).toBe(409)  
  expect(UsuarioRepository.criar).not.toHaveBeenCalled()  
})
```

O service lança o erro 409, e o `not.toHaveBeenCalled()` confere que o `criar` nem foi chamado: nada foi salvo.

Login: senha errada (401)

```
beforeEach(async () => {
  vi.resetAllMocks()
  const senhaHash = await bcrypt.hash('123456', 4)
  const ana = new Usuario({ nome: 'Ana', email: 'ana@email.com',
    senhaHash })
  UsuarioRepository.buscarPorEmailComSenha.mockResolvedValue(ana)
})

it('senha errada responde 401', async () => {
  const res = await request(app)
    .post('/api/auth/login')
    .send({ email: 'ana@email.com', senha: 'errada' })

  expect(res.status).toBe(401)
  expect(res.body.message).toBe('Email ou senha incorretos.')
})
```

O hash é de verdade: o login usa o `bcrypt.compare` real.

Rota protegida com token de verdade

```
const ana = new Usuario({ nome: 'Ana', email: 'ana@email.com' })
const id = ana._id.toString()
const token = jwt.sign({ id, email: ana.email }, process.env.JWT_SECRET)

it('devolve o usuário do token', async () => {
  UsuarioRepository.buscarPorId.mockResolvedValue(ana)

  const res = await request(app)
    .get('/api/usuarios/perfil')
    .set('Authorization', `Bearer ${token}`)

  expect(res.status).toBe(200)
  expect(res.body.usuario.nome).toBe('Ana')
  expect(UsuarioRepository.buscarPorId).toHaveBeenCalledWith(id)
})
```

O `toHaveBeenCalledWith(id)` confere que a API buscou o id do token.

O segredo do JWT no teste

Arquivo `vitest.config.js`, na raiz do projeto:

```
import { defineConfig } from 'vitest/config'

export default defineConfig({
  test: {
    env: {
      JWT_SECRET: 'segredo-so-para-testes',
      BCRYPT_SALT_ROUNDS: '4',
    },
  },
})
```

env → variáveis que valem só durante os testes.

JWT_SECRET → sem ele, o `getToken` lança erro 500.

SALT_ROUNDS → 4 em vez de 10: o `bcrypt` fica bem mais rápido nos testes.

.env → o teste não lê o `.env` (quem carrega é o `server.js`): roda igual em qualquer computador.



Cuidado: mock que vaza para o próximo teste

Sem zerar os mocks, o valor de um teste **continua valendo** no próximo: o teste passa sozinho e falha junto com os outros.

- O segundo teste recebeu a Ana do teste anterior: para a API, o e-mail já existia.
- Com `beforeEach(() => vi.resetAllMocks())`, cada teste começa do zero, e a ordem deixa de importar.

```
it('email repetido responde 409', ...) // mock: a Ana
it('cadastra a Bia', ...) // sem mock do buscarPorEmail
// AssertionError: expected 409 to be 201
```



Cuidado: não teste o mock

Se o teste só confere o que você mesmo mandou o mock devolver, ele **não testa nada** da API.

- Bom: mockar o repository e conferir o que a API fez com os dados: status, senha fora do body, token.
- Ruim: mockar o service inteiro e conferir que a rota devolveu o que o mock devolveu.
- O mock não pega erro do banco (**unique**, validação do Schema): para isso, um banco de teste.



Boas práticas

Um bom teste



Nome que explica

"senha errada responde 401": quem lê o nome já sabe o que quebrou.



Um comportamento por teste

Se falhar, você sabe exatamente o quê.



Independente

Roda sozinho, em qualquer ordem, quantas vezes for preciso.



Prepara, age, confere

Configura os mocks, faz a requisição e confere a resposta. Nessa ordem.

O que testar em cada rota

200 201 Deu certo

Status, body e o que não pode aparecer, como a senha.

400 Dados inválidos

Campo faltando ou valor errado.

401 Sem login

Sem token, formato errado, token falso.

403 Sem permissão

Usuário comum numa rota de admin.

404 Não encontrado

ID que não existe ou usuário apagado.

409 Conflito

E-mail repetido.

Resultado final

```
$ npm test
```

```
RUN v5.0.2
```

- ✓ tests/login.test.js (2 tests)
- ✓ tests/cadastro.test.js (2 tests)
- ✓ tests/validacao.test.js (2 tests)
- ✓ tests/perfil.test.js (2 tests)
- ✓ tests/autenticacao.test.js (3 tests)
- ✓ tests/app.test.js (2 tests)
- ✓ tests/criarErro.test.js (1 test)

```
Test Files 7 passed (7)
```

```
Tests 14 passed (14)
```

14 testes → cadastro, login, perfil, validação e autenticação em poucos segundos, sem abrir o Postman.

antes do push → rode **npm test** antes de cada **git push**: se algo quebrou, você descobre antes do Render.

1. No seu projeto do boilerplate, instale vitest e supertest e crie os scripts test e test:watch.
2. Crie o vitest.config.js com um JWT_SECRET de teste.
3. Teste sem banco: GET /, rota inexistente (404), cadastro sem e-mail (400) e perfil sem token (401).
4. Com o repository mockado, teste o cadastro (201, sem senhaHash) e o login com senha errada (401).
5. Faça um teste falhar de propósito e leia o Expected e o Received.



Dica: comece copiando um teste que já passa e mude uma coisa só. O npm run test:watch roda de novo a cada arquivo salvo.



Desafio: teste o PATCH /api/usuarios/perfil: nome trocado (200) e senha curta (400). Bônus: se você fez o papel admin, teste o 403.



Documentando a API com Swagger

Quem consome a API precisa saber quais rotas existem, o que mandar e o que volta.

Por que documentar a API?



Quem consome a API

No Módulo 11, a tela do FlutterFlow precisa saber a rota, o body e a resposta de cada chamada.



Um contrato

A documentação diz o que a API promete: o que entra, o que sai e cada erro possível.



Testar no navegador

O Swagger UI tem o botão Try it out: manda a requisição direto da página.



OpenAPI e Swagger

OpenAPI é o padrão do arquivo. Swagger é o nome das ferramentas que leem esse padrão.

Instalando e ligando no app.js

No terminal: `npm install swagger-ui-express`. A documentação fica num arquivo JavaScript comum: `src/docs/openapi.js`.

```
import swaggerUi from 'swagger-ui-express'
import openapi from './docs/openapi.js'

// junto das outras rotas, antes do 404
app.get('/api/docs.json', (req, res) => res.json(openapi))
app.use('/api/docs', swaggerUi.serve, swaggerUi.setup(openapi))
```

`/api/docs`: a página para abrir no navegador. `/api/docs.json`: o mesmo conteúdo em JSON, que o Postman importa.

Anatomia do openapi.js

```
const openapi = {  
  openapi: '3.0.3',  
  info: {  
    title: 'API Boilerplate Lions Dev',  
    version: '1.0.0',  
  },  
  servers: [{ url: 'http://localhost:3000' }],  
  components: {  
    securitySchemes: {  
      bearerAuth: { type: 'http', scheme: 'bearer' },  
    },  
  },  
  paths: {  
    // uma chave para cada rota  
  },  
}  
  
export default openapi
```

info → nome e versão que aparecem no topo da página.

servers → para onde o Try it out manda as requisições. Depois do deploy, acrescente a URL do Render.

securitySchemes → avisa o Swagger que a API usa token Bearer.

paths → as rotas: quase todo o trabalho fica aqui.

Cadastro: o que a rota recebe

```
'/api/auth/cadastro': {  
  post: {  
    summary: 'Cadastra um usuário e devolve o token',  
    requestBody: {  
      required: true,  
      content: {  
        'application/json': {  
          schema: {  
            type: 'object',  
            required: ['nome', 'email', 'senha'],  
            properties: {  
              nome: { type: 'string', example: 'Ana' },  
              email: { type: 'string', example: 'ana@email.com' },  
              senha: { type: 'string', example: '123456' },  
            },  
          },  
        },  
      },  
    },  
  },  
},
```

O required marca o que não pode faltar. O example vira o body pronto no Try it out.

Cadastro: o que a rota devolve

```
responses: {  
  201: { description: 'Usuário criado' },  
  400: { description: 'Campo faltando' },  
  409: {  
    description: 'Email já cadastrado',  
    content: {  
      'application/json': {  
        example: { message: 'Email já cadastrado.' },  
      },  
    },  
  },  
}
```

um status por chave →
documente os erros
também: o frontend
precisa saber o que
mostrar em cada caso.

example → a mensagem
que a API manda de
verdade, copiada do
middleware ou do
service.

Repetiu muito? Uma função como erro(mensagem) monta esse bloco para você.

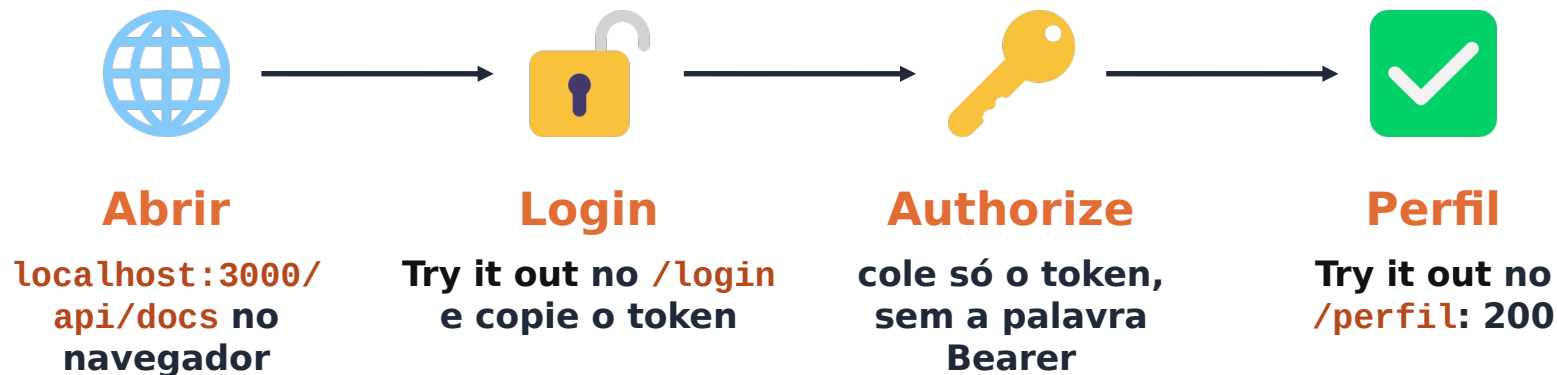
Rota protegida: o cadeado

```
'/api/usuarios/perfil': {  
  get: {  
    tags: ['Usuários'],  
    summary: 'Mostra o usuário do token',  
    security: [{ bearerAuth: [] }],  
    responses: {  
      200: { description: 'Usuário logado' },  
      401: { description: 'Sem token ou token inválido' },  
    },  
  },  
},
```

security → coloca um cadeado na rota. O nome **bearerAuth** é o mesmo do **securitySchemes**.

Authorize → botão no topo da página: cole o token do login uma vez e ele vai em todas as rotas com cadeado.

Testando pelo Swagger UI



O Swagger UI acrescenta o Bearer sozinho, porque o esquema é scheme: "bearer".



Cuidado: a documentação no lugar errado

Registrada **depois** do 404, a documentação nunca aparece: a requisição cai no 404 antes de chegar lá.

- A ordem do **app.use** importa, como a ordem dos middlewares no Módulo 9.
- Ponha **/api/docs** junto das rotas da API, antes do 404 e do middleware de erro.
- Sem a barra final, **/api/docs** redireciona (301) para **/api/docs/**. É normal.



Cuidado: a documentação que mente

Mudou a rota e esqueceu a documentação? O frontend confia nela e **quebra. Documentação desatualizada é pior que nenhuma.**

- Mudou uma rota, um campo ou uma mensagem? Atualize o **openapi.js** no mesmo commit.
- Um teste pode conferir se cada rota documentada existe de verdade (próximo slide).

Um teste para a documentação

```
import openapi from '../src/docs/openapi.js'

describe('a documentação não pode mentir', () => {
  for (const [rota, metodos] of Object.entries(openapi.paths)) {
    for (const metodo of Object.keys(metodos)) {
      it(`${metodo.toUpperCase()} ${rota} existe`, async () => {
        const res = await request(app)[metodo](rota).send({})

        expect(res.status).not.toBe(404)
      })
    }
  }
})
```

Um it para cada rota do openapi.js. Documentou uma rota que não existe? O teste fica vermelho.

Uma documentação, três usos



Swagger UI

Testar as rotas no navegador, sem instalar nada.



Postman

Import → Link → <http://localhost:3000/api/docs.json>: a coleção sai pronta, com todas as rotas.



README e projeto final

Coloque o link da documentação no README: quem avalia testa a API na hora.



Debug da API com breakpoints

O teste diz que algo quebrou. O debugger mostra por quê.

console.log x breakpoint

Com console.log

```
function validarCadastro(req, res, next) {  
  console.log('body:', req.body)  
  const { nome, email, senha } = req.body  
  // ...  
}  
  
// muda o código, reinicia o servidor,  
// repete... e depois lembra de apagar
```

Com breakpoint

- `const { nome, email, senha } = req.body`

VARIABLES

```
req.body: undefined  
req.method: "POST"  
req.url: "/cadastro"
```

nenhuma linha nova no código

O breakpoint pausa o programa na linha e mostra todas as variáveis, sem mudar o código.

Debugar um servidor é diferente



O programa não termina

Um script roda e acaba; o servidor fica esperando. O breakpoint só para quando uma requisição passa pela linha.



Quem dispara é o cliente

Mande a requisição pelo Postman (ou pelo teste) e o VS Code para na linha marcada.



Pausa tudo

Enquanto está parado, a API inteira espera, inclusive as requisições dos colegas.

Três jeitos de ligar o debugger



Debug Terminal

Na setinha ao lado do + do terminal, abra um JavaScript Debug Terminal e rode `npm start`. O mais simples.



launch.json

Uma configuração salva no projeto: aperte F5 e pronto. Próximo slide.



--inspect

`node --inspect src/server.js` e depois conectar o editor. Funciona em qualquer lugar.

Os três fazem a mesma coisa: ligam o inspector do Node e conectam o VS Code a ele.

Configuração salva: .vscode/launch.json

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "type": "node",
      "request": "launch",
      "name": "Debug da API",
      "program": "${workspaceFolder}/src/server.js",
      "envFile": "${workspaceFolder}/.env",
      "skipFiles": ["<node_internals>/**"]
    }
  ]
}
```

program → o arquivo que sobe a API, o mesmo do **npm start**.

envFile → carrega o **.env** antes de rodar: **MONGO_URI** e **JWT_SECRET** chegam certos.

skipFiles → não entra no código interno do Node quando você usa o Step Into.

Pelo terminal: node --inspect

```
$ node --inspect src/server.js
```

```
Debugger listening on ws://127.0.0.1:9229/ae0dca4a-29f2-...
```

```
For help, see: https://nodejs.org/en/docs/inspector
```

```
Servidor rodando na porta 3000.
```

```
Debugger attached.
```

Para conectar: Ctrl+Shift+P → Debug: Attach to Node Process (ou chrome://inspect). O --inspect-brk já para na primeira linha.

Onde pôr o breakpoint



middleware

o token chegou? o
body veio?

controller

o que o cliente
mandou

service

a regra de negócio

repository

o que o banco
devolveu

Não sabe onde está o erro? Pare no começo do caminho e avance com o Step Over até achar o valor errado.

Os controles do debugger

F5 Continue

segue até o próximo breakpoint.

F10 Step Over

executa a linha e para na seguinte.

F11 Step Into

entra na função chamada nesta linha.

Shift+F11 Step Out

termina a função e volta para quem chamou.

Variables e Watch

o valor de cada variável; no Watch, você escolhe o que acompanhar.

Debug Console

digite `req.body.email` e veja o valor na hora, com tudo parado.

Breakpoints que pensam



Condicional

Clique direito na margem → Add Conditional Breakpoint:
`req.body.email === "ana@email.com"`. Só para quando for verdade.



Logpoint

Add Logpoint com Login de `{req.body.email}` : escreve no Debug Console sem mexer no código.



Hit Count

Para só na enésima passagem pela linha. Bom dentro de laços.

Caso real: o teste achou um 500

Cadastro sem body nenhum: a validação deveria responder 400...

```
it('sem body responde 400, não 500', async () => {  
  const res = await request(app)  
    .post('/api/auth/cadastro')  
  
  expect(res.status).toBe(400)  
})
```

Saída do Vitest

```
$ npm test  
FAIL tests/corpoVazio.test.js  
AssertionError: expected 500 to be 400  
  
- Expected  
+ Received  
  
- 400  
+ 500
```

Parando no validarCadastro

Breakpoint na desestruturação

```
function validarCadastro(req, res, next) {  
  // • breakpoint nesta linha  
  const { nome, email, senha } = req.body  
  if (!nome) {  
    return next(criarErro("...", 400))  
  }  
}
```

Painel Variables

```
req.body: undefined  
req.headers:  
  content-type: undefined  
req.method: "POST"  
req.url: "/cadastro"  
req.originalUrl: "/api/auth/cadastro"
```

No Express 5, sem body, o req.body fica undefined. Desestruturar undefined gera um erro, e o middleware de erro responde 500.

A correção

```
// antes: sem body, TypeError → middleware de erro → 500  
const { nome, email, senha } = req.body
```

```
// depois: sem body, nome é undefined → validação → 400  
const { nome, email, senha } = req.body ?? {}
```

O ?? troca undefined (ou null) por um objeto vazio. A validação roda, responde 400, e o teste fica verde.



Cuidado: o breakpoint que nunca para

Bolinha cinza e vazada no lugar da vermelha: o VS Code não está conectado ao programa, ou a linha nunca foi executada.

- Rode pelo JavaScript Debug Terminal, pelo F5 ou com `--inspect`. Terminal comum não para em breakpoint.
- Confira se a requisição chega mesmo nessa rota: método e caminho certos no Postman.
- O breakpoint precisa estar no arquivo que o servidor importa, não numa cópia.



Cuidado: debugger em produção, nunca

Quem se conecta ao inspector pode **rodar qualquer código** no servidor. No Render, o start continua sendo `node src/server.js`.

- Nunca use `--inspect=0.0.0.0`: abre a porta 9229 para a rede inteira.
- Uma linha `debugger`; esquecida não faz nada sem debugger conectado, mas não deixe ir para o GitHub.
- Parou no breakpoint? A API inteira está pausada: solte com F5 antes de sair.

Debugando um teste



No Debug Terminal

Rode `npx vitest run tests/login.test.js`: os breakpoints no teste e no service funcionam.



Um teste só

Troque `it` por `it.only` para rodar só o teste que falha. Volte para `it` antes do commit.



Teste + debugger

O teste reproduz o erro sempre igual; o debugger mostra o porquê. Juntos, acham qualquer bug.

Atividade Prática

Swagger e debug

1. **Instale o `swagger-ui-express` e documente cadastro, login e perfil em `src/docs/openapi.js`.**
2. **Abra `/api/docs`, faça login pelo Try it out e use o Authorize para chamar o perfil.**
3. **Crie o `.vscode/launch.json` e ponha um breakpoint no `validarCadastro`.**
4. **No Postman, mande o cadastro com Body none e descubra pelo painel Variables por que a API responde 500.**
5. **Corrija, escreva o teste do corpo vazio e rode `npm test`.**



Dica: no Postman, a opção none da aba Body manda a requisição sem corpo nenhum.



Desafio: importe o `/api/docs.json` no Postman (Import → Link) e compare com a coleção que você montou à mão.

Resumo da aula



Vitest

describe, it e expect: o teste confere se a saída é a esperada.



supertest

request(app) testa a rota inteira, sem servidor e sem porta.



Mock

vi.mock() troca o repository: sem banco, o resto roda de verdade.



Independência

vi.resetAllMocks() no **beforeEach**: cada teste começa do zero.



Swagger

openapi.js + swagger-ui-express: a API documentada e testável em **/api/docs**.



Debug

O teste mostra que quebrou; o breakpoint mostra por quê.

Desafio: testes do flashcards

Fio condutor

Escreva testes para as rotas de baralhos criadas no desafio do Módulo 9.

- GET /api/baralhos sem token responde 401.
- POST /api/baralhos sem título responde 400 (valide no começo da rota ou num middleware).
- Com o repository mockado e um token de verdade, o cadastro responde 201.
- A listagem chama o repository com o id do usuário do token: use toHaveBeenCalledWith.



Dica: copie a estrutura do perfil.test.js: ele já gera um token com o JWT_SECRET do vitest.config.js.



Desafio: teste que um usuário não consegue apagar o baralho de outro.

Material do módulo



Aponte a câmera do celular

[github.com/nicolassmotta/curso-lions-dev/
tree/main/modulo09b](https://github.com/nicolassmotta/curso-lions-dev/tree/main/modulo09b)



conteudo/
teoria comentada e o cheat sheet do módulo



lista_de_exercicios/
listas para praticar em casa



exercicios_resolvidos/
gabaritos, para conferir depois de tentar



Fim!

No Módulo 10, a API, agora testada, vai para a internet com o deploy no Render.